

AI Software Course — Study Notes & Revision Guide

Course: *Coding & Learning: Using GenAI To Support Software Development*

(WeThinkCode_) **Purpose:** A condensed, test-ready reference covering the key points of every section. Use it to revise the core ideas, prompt strategies, pitfalls, and principles without re-reading the full course.

The Big Picture

Generative AI (GenAI) creates new content — text, code, images, audio — by learning patterns from existing data. The core technology is the **Large Language Model (LLM)** (e.g. Claude, ChatGPT, GitHub Copilot), trained on vast text to understand and generate language and code.

The course's central message: **treat AI as an augmentation of your skills — like Iron Man's suit — not a crutch.** The value comes from asking the right questions, providing good context, verifying results, and staying the active learner.

The course has four parts:

1. **A brief overview of GenAI tools** — the landscape and how to choose.
 2. **Using AI for everyday work** — comprehension, documentation, debugging, testing, refactoring.
 3. **Learning with AI** — deepening your current language, learning a new language, learning a new framework/library/API.
 4. **Principles when using AI** — the recurring principles and practices that tie it all together.
-

Module 1 — A Brief Overview of Generative AI Tools

Key idea: There are many tools; you only need to understand the categories and pick the right tool for the job.

Tool categories

- **Code Assistant tools** — suggest code as you type, integrated into your IDE. Examples: GitHub Copilot, Amazon CodeWhisperer, Tabnine, Cursor. Increasingly they also offer chat interfaces.
- **Code Explanation & Learning Assistants** — explain unfamiliar code and concepts at your level. Examples: Claude (strong at reasoning/explaining), ChatGPT (versatile coding knowledge), Gemini (web-connected).
- **Prototyping tools** — turn ideas/descriptions into working prototypes. Examples: Bolt, v0, Replit, Lovable.

Legal & security considerations (know these): data privacy (prompts/code may be stored and used for training), intellectual property & licensing of generated code, code attribution, security scanning of generated code, on-premise vs. cloud processing, terms of service, export controls, and cost structure. Always follow your organization's policy on which tools are allowed.

Recommended for this course: **GitHub Copilot** for code assistance + **Claude** for concept explanation and problem-solving. Most exercises work on the free tiers.

Module 2 — Using AI for Everyday Work

2.1 Comprehending existing codebases

Most work happens on **existing** code, so comprehension is a core skill. Three levels:

1. **Knowing where to start** — build a mental map: identify the entry point, tooling/dependencies, project structure, and how features map to code. Key prompts: *Understanding Project Structure and Technology Stack, Finding Feature Implementation Locations, Understanding Domain Models and Business Concepts.*
2. **Understanding existing code functionality** — understand *before* changing. Prompts: *Understand how a specific feature works, Deepen understanding of a codebase* (AI asks you guiding questions instead of explaining), *Mapping Data Flow and State Management.*
3. **Deciphering complex functions & algorithms** — break down dense/cryptic code. Prompts: *Step-by-Step Analysis, Decipher Code with Unclear Intent, Understand Complex Logic and Control Flow.*

Pitfalls: being too vague about your current understanding; jumping to implementation before understanding purpose (ask "why" before "how"); not answering the AI's questions back to you; trying to understand everything at once (work outward in concentric circles from your task).

Section principle → Communicate Clearly: every good prompt gives clear, specific asks plus rich context.

2.2 Generating & improving documentation

Three documentation targets, each with three prompts:

- **Code documentation** — *Comprehensive Function Documentation* (description, params, return, exceptions, examples, edge cases), *Intent and Logic Explanation*, *Documentation Style Conversion.*
- **API documentation** — *Endpoint Documentation Generation*, *API Reference Conversion* (e.g. Markdown → OpenAPI/Swagger), *API Usage Guide Creation* (audience- and tone-specific).
- **User guides & README files** — *Project README Generation*, *Step-by-Step Guide Creation*, *FAQ Document Generation.*

Key point: Generate, then **review for accuracy and completeness**, iterate the prompt, and combine the best of multiple outputs. Documentation quality depends on the context you supply.

2.3 Debugging errors

Three focus areas:

1. **Tracing error messages & stack traces** — *Error Message Translation* (plain-language meaning), *Root Cause Analysis*, *Dependency and Version Tracing*. Aim to understand the concept, not just the one-line fix.
2. **Identifying performance bottlenecks** — *Slow Code Analysis*, *Memory Usage Optimization*, *Slow Database Query Analysis*. Always **measure before and after**; ask for the underlying performance concepts and profiling tools.
3. **Limitations & verification** — *Collaborative Solution Verification*, *Learning Through Alternative Approaches*, *Developing a Critical Eye*. AI can hallucinate; treat its answers as hypotheses to test.

Section principle → Verify! Continuously test AI output against tests, logging, docs, and other humans.

2.4 Helping with testing

Learn testing *skills*, not just generate tests. Three areas:

1. **Understanding what to test** — analyse behaviours and edge cases; plan tests by priority (unit vs. integration).
2. **Improving a single test** — make a test check *behaviour* not implementation details; sharpen assertions; cover edge cases.
3. **Test-Driven Development (TDD)** — red/green/refactor: write a failing test, write minimal code to pass, refactor, repeat. Also use TDD to reproduce and fix bugs (write a failing test that demonstrates the bug first).

Pitfalls: letting AI write all your tests; testing everything at once; chasing test *quantity* over *quality*; skipping the learning for convenience.

Section principle → Learning Through Guided Practice: ask AI to *guide* (question and give feedback) rather than *generate*.

2.5 Refactoring & enhancing code

Four areas:

1. **Understanding what to change** — *Code Readability Improvement*, *Function Refactoring*, *Code Duplication Detection*.
2. **Breaking down complex functions** — *Function Responsibility Analysis*, *Single-Responsibility Extraction*, *Conditional Logic Simplification*.
3. **Improving readability** — better variable/function names, comments/docs, and structure.

4. **Implementing design patterns** — recognise opportunities for **Strategy** (conditional logic), **Factory** (complex object creation), **Observer** (one change → many updates), **Adapter** (incompatible interfaces).

Golden rule of refactoring: always have tests in place *before* refactoring real code, and verify behaviour is preserved after.

Section principle → Know your words: you must know the vocabulary (refactoring, decomposition, single responsibility, design patterns) to ask about and apply these ideas.

Module 3 — Learning with AI

3.1 Deepening your current programming language

Use AI to level up in a language you already use:

- **Idiomatic code** — rewrite your code to follow language best practices and use features you're under-using.
- **Code quality enhancement** — review older code for code smells; rate readability, performance, maintainability; build a personal review checklist.
- **Advanced language features** — learn features you avoid (e.g. Python decorators, JS generators) with examples, use cases, a practice project, and common mistakes.

Habit: journal 3 key learnings per activity.

3.2 Learning a new programming language

Structured approach:

- **Create a learning journey plan** — phases, prerequisites, steps, and verification activities, driven by 2-3 concrete goals.
- **Four-step prompting strategy:** (1) **Conceptual Understanding** — philosophy, problems the language solves, mental-model shifts, misconceptions; (2) **Step-by-Step Breakdown** — a specific concept vs. its equivalent in your source language; (3) **Guided Implementation** — build a small example with syntax explained; (4) **Understanding Verification** — write it yourself, then have AI check best practices and flag source-language habits.
- **Advanced techniques:** Using Context Effectively, Promoting Deep Understanding, Learning Through Teaching, Using a Tutor (AI asks questions and gives hints, not answers).
- **Build technical vocabulary** and finish with a mini-project.

3.3 Learning a new framework, library or API (FastAPI as the worked example)

Strategies to accelerate the learning curve:

- **Getting started quickly** — fundamentals, a "Hello World", then structure and validation.
- **Contextual learning** — map the new tech onto frameworks you already know (build a "translation table"); understand *why* it was designed that way (e.g. FastAPI's use of Pydantic, type hints, async-first, automatic docs).
- **Documentation navigation** — use AI to plan a reading order, deep-dive complex features, and translate abstract docs into concrete code.
- **Understanding code patterns** — decode advanced patterns (repository, dependency injection, middleware, decorators) and trace request execution flow.

Set expectations: AI is a *learning partner*, not an oracle — verify AI-provided info (models can be out of date on newer tech), watch for common misconceptions, and balance AI help with your own understanding.

Module 4 — Principles When Using AI (the heart of the course)

The 5 Principles

1. **Communicate Clearly** (*from Comprehension*) — talk to AI like a knowledgeable colleague who lacks your context. Be specific and give enough context. Use the **SCSD** approach: **S**pecific, **C**ontextual, **S**tructured, **D**escriptive.
2. **Verify!** (*from Debugging*) — AI is fallible and can hallucinate. Verify by restating in your own words, asking for trade-offs/alternatives, and checking with other humans and sources.
3. **Learning Through Guided Practice** (*from Testing*) — have AI ask questions and give feedback rather than generate complete solutions, so you build lasting skills instead of dependency.
4. **Know Your Words** (*from Refactoring*) — you need software-engineering vocabulary to ask precise questions and understand answers (and to communicate with your team).
5. **Drive Your Own Learning** (*from Learning a New Language*) — you are the active learner; set clear objectives, never accept solutions you don't understand, and use AI (and humans) to test your understanding.

The 5 Practices

1. **Define learning outcomes** — set a clear goal ("understand how authentication works") not a vague one ("explain code"); use a learning journey map for big topics.
2. **Personal prompt template library** — save your most effective prompts ("magic spells") and share them with teammates.

3. **Document learnings** — write insights in your own words (a developer journal) to cement understanding and build a personal knowledge base.
4. **Create an AI response checklist** — evaluate AI output: Is it secure? Does it meet our standards? What edge cases are missed? Share it with the team.
5. **Test with humans** — always get experienced teammates to review important AI-generated solutions.

Cheat Sheet — Prompt Strategies by Task

Task	Go-to prompt approach
Understand an unfamiliar project	State your best guess + paste structure/config → ask AI to validate, name entry points, and list questions to ask the team
Understand a feature/algorithm	Paste code + your understanding → ask for plain-language explanation, execution walkthrough, and a mental model; ask it to quiz you
Document code/API/README	Ask for structured docs (params, returns, errors, examples), then convert format and add a usage guide/FAQ
Debug an error	Paste error + code → ask for plain-language meaning, root cause, fix, and prevention concepts
Optimise performance	Paste code + context (input size, environment) → ask why it's slow, 2-3 improvements, and profiling tools; measure before/after
Verify an AI solution	Ask it to trace a concrete example, propose alternatives, and suggest edge-case tests — don't trust, verify
Write tests	Ask AI to question you about behaviours/edge cases and coach TDD, not to write all tests
Refactor	Ask for responsibilities, a decomposition plan, and a behaviour-preserving rewrite — with tests first
Learn a new language/tech	Four-step strategy: concept → breakdown → guided build → verify; map onto what you already know

Cheat Sheet — Common Pitfalls to Avoid

- Being vague or giving incomplete context.
 - Accepting AI output without verifying it (hallucinations, outdated info, insecure patterns).
 - Using AI to *generate* answers instead of to *learn* — creating dependency.
 - Trying to understand or test everything at once instead of focusing.
 - Refactoring without tests; optimising without measuring.
 - Not knowing the terminology needed to ask precise questions.
-

Quick Self-Test (cover the answers)

1. **What does SCSD stand for, and which principle does it support?** — Specific, Contextual, Structured, Descriptive; supports *Communicate Clearly*.
 2. **Name the five principles.** — Communicate Clearly; Verify!; Learning Through Guided Practice; Know Your Words; Drive Your Own Learning.
 3. **Which four design patterns are highlighted, and their triggers?** — Strategy (conditionals), Factory (complex creation), Observer (one→many updates), Adapter (incompatible interfaces).
 4. **What are the three comprehension levels?** — Knowing where to start; understanding existing functionality; deciphering complex functions/algorithms.
 5. **What is the four-step language-learning strategy?** — Conceptual Understanding → Step-by-Step Breakdown → Guided Implementation → Understanding Verification.
 6. **What's the golden rule before refactoring real code?** — Have tests in place and verify behaviour is preserved.
 7. **Why "guided practice" over "generation" in testing?** — It builds lasting skills and critical thinking instead of dependency.
 8. **Two recommended tools for the course?** — GitHub Copilot (assistance) and Claude (explanation/problem-solving).
 9. **Why must you always verify AI output?** — AI is fallible, doesn't think like us, and can hallucinate; verify with your own words, alternatives, other humans and sources.
 10. **What is the course's core metaphor for using AI well?** — An augmentation of your abilities (Iron Man's suit), not a crutch.
-

Source: WeThinkCode_ "Coding & Learning: Using GenAI To Support Software Development" — <https://ai.wethinkco.de/ai-software/> (accessed via the ai.wethinkcode.co.za account).